

TekkForge & Omnitribe

Two projects for the KORG Electribe 2 Sampler: **TekkForge** turns songs, sample folders and ideas into playable pattern sets and talks to the instrument from the computer. **Omnitribe** extends the instrument's own firmware so that new functions run *inside* the device. This document says what works today, what is being tested right now, and what comes next.

9 modules in one app (TekkForge)	21 firmware modules that load and run on the device	16 MB of the device's flash memory mapped and backed up	7,500+ automated tests across both projects
--	---	---	---

WORKS ON THE DEVICE

 measured on real hardware

BUILT, IN TEST

compiles and is verified on the computer, first hardware run pending

PLANNED

 designed, not yet built

TekkForge IN DAILY USE Pattern editor, song-to-set generator, sample bank workshop, MIDI control, effect presets, firmware workbench, device mirror and memory access.	Omnitribe MODULES RUN INSIDE THE DEVICE A small add-on inside the firmware that loads plug-in modules over USB and runs them. Now drives modules from the device's own timer so they make sound on their own; next: reaching them from the pads and internal sequencer.	Bootloader BUILT, FIRST RUN PENDING Start a firmware from the SD card without flashing; flash safely from the device's own menu. Four improvements built; first run over the debug connector.
---	--	--

1 · The instrument in one paragraph

The Electribe 2 Sampler is a groove box: sixteen "parts", each with a sample or a synth sound, a step sequencer with 64 steps, effects, and 250 pattern slots. It stores its firmware and its factory sounds in a 16 MB flash chip, boots through a small loader, and talks to a computer over USB-MIDI. The community project *hacktribe* unlocked the sampler's hidden synthesizer engine years ago; both of our projects build on that groundwork and give credit for it in the documentation.

2 · TekkForge — what it does today WORKS

From a song

Measure tempo and key, separate melody, bass, drums and vocals, cut single hits, and build a bank with matching patterns — all on the local machine.

From a folder

A directory full of samples is sorted, fitted into the device's memory budget and assembled into a playable bank.

By hand

The pattern editor builds patterns step by step: notes, velocity, length, chords, own sounds. A library keeps and searches patterns.

To the device

Finished files go to the SD card or straight into a slot over MIDI while the running pattern stays untouched. Pads can be freely assigned; external controllers can drive parts.

Effects and grooves

288 ready effect presets, own presets and swing templates — including a groove extracted from a song. Works with the extended hacktribe effect set.

Device mirror and workbench

The device reports every knob and button; TekkForge mirrors that state and offers a workbench for the undocumented parts of the MIDI protocol.

Memory access and backups

Read the device's working memory, take a complete backup of the 16 MB flash, compare two readings and name the differences. A device monitor shows values with names instead of raw numbers.

Firmware workbench

Check and build firmware update files, swap the start screen, extend the oscillator table and the modulation types, convert between the synth and the sampler firmware — with the device's product check handled.

Two firmware worlds are recognised automatically: the factory firmware and the extended hacktribe version. Functions that only work on one of them stay locked on the other. Everything runs locally; nothing leaves the computer except an optional link import and an optional AI request.

New this month: the firmware side of TekkForge

- **Client for the Omnitribe add-on:** identity, firmware information, live telemetry and parameter read/write of the running device, plus scripts that upload modules, run them and read the results back.
- **Bootloader tools:** build and check boot sectors and `BOOT.VSB` update files, prepare an SD card for the bootloader, start the bootloader over USB (firmware upgrade mode) or, on

suitable connections, over MIDI. Every one of these actions needs a typed confirmation and never flashes by itself.

3 · Omniribe — new functions inside the firmware

What it is. The device's firmware is a fixed 2 MB program. Omniribe adds a small add-on to it (about 10 KB) that is reached over normal USB-MIDI with its own message format. The add-on can report what the device is doing, read and change parameters live, read any memory location, control transport — and, since this month, **load plug-in modules and run them**. Modules are small native programs (arpeggiator, chord player, modulation matrix, sidechain, voice management, and more; 21 in total) that are sent over USB, placed in free memory and started while the instrument keeps playing. Nothing in the flash changes for that; switching the device off removes everything again.

What works on the device MEASURED

CAPABILITY	SINCE
Talk to the add-on over USB-MIDI: identity, firmware information, telemetry, read and set parameters of the sixteen parts, transport (play/stop, clock)	August 2026
Feed messages into the firmware's own MIDI input path — the add-on can "play" the instrument from inside (control changes audible)	August 2026
Read any memory location while the device runs; complete map of the memory; the firmware's interrupt table read live	September 2026
Upload a module in chunks, validate it, place it in free memory, read it back	17 Sept 2026
Run modules: a test module and the modulation matrix, arpeggiator and chord player start and return; all 21 modules initialise; single callbacks (clock tick, note on, parameter change) can be triggered from the computer and their internal state read back	18 Sept 2026

Getting modules to run took one false start: the first attempt froze the device. The causes were found without guesswork — the module code was placed at a different address than it was built for, and the processor still had old instructions in its cache — and both are fixed in a way that is checked automatically at build time.

Event routing MEASURED ON THE DEVICE

Until now a module only did something when the computer poked it. This build connected the modules to the instrument itself, and the pieces below were checked on real hardware:

- **Sound out (works):** what a module wants to play goes into the firmware's own input path, so the instrument plays it like any incoming MIDI. Arpeggiator, chord player and a modulation-matrix filter sweep were all heard coming out of the device.
- **Timing (works):** a periodic tick from the firmware's own timer, installed and removed at runtime, drives arpeggiators and modulation; the tick runs steadily with no skips.
- **MIDI in (needs a different hook):** system-exclusive messages reach the modules, but ordinary notes and controllers sent from the computer do *not* arrive at the current hook point — the device routes those elsewhere first. Until that hook is added, the modules are driven through direct commands from the computer; external-keyboard input to the modules is still open.
- **Safety:** every stage has counters, every stage can be switched off again, and a power cycle restores the plain firmware. A review this week fixed the corner cases found on paper (ordering during module load, state surviving a restart, a hanging arpeggio note, chords never releasing, a modulation flood).

What the device test showed: the timer tick runs steadily; the modulation matrix runs by itself; the arpeggiator, the chord player and a filter sweep on part 1 were all heard; every stage switched back off cleanly. External notes and controllers from the computer did not reach the modules at the current hook, so that ingress point is the remaining piece.

What comes next PLANNED

- **Standalone use:** today the modules are only pumped when MIDI arrives or the computer asks; a hook in the firmware's main loop makes them run without a computer, and later draws module pages on the display.
- **Modules built into the firmware file** so they are there after every power-on, with the timer installed automatically.
- **Pads and the internal sequencer** reaching the modules (today only external MIDI does), via the device's own MIDI output.
- **A tidy control map** before any on-device user interface: some modules share addresses or use ones that cannot be sent over MIDI.
- **Module interface v2** with pitch bend, aftertouch and controller input for the expressive modules.

4 • Firmware builds in use

- **MOD132 / OSZ88:** the hacktribe sampler firmware, extended so its modulation-type table and the code around it carry 132 entries instead of 72, plus 88 additional oscillator-table entries. The extended modulation types (72–131) are now **settable, audible and correctly named on the device** via Omnitribe: the add-on calls the firmware's own mod-type setter, so all 132 types are reachable live even though the panel's own encoder knob still stops at

72 (confirmed 18 Sept — mod type 120, which the panel numbers 121, shows "S&H Filter" and sounds distinct). The 88 extra oscillator entries are likewise heard and confirmed.

- **Synth ↔ Sampler crossgrade** confirmed on the device with the product check handled; the factory sound file remains the boundary between the two.
 - **Flash backup and finding:** a full 16 MB dump showed that a test sound file from an earlier bricking experiment was still in the flash — which explained a long-standing "sounds wrong" mystery and gave a clear restore route. The device was recovered twice over the debug connector (JTAG), so that safety net is proven.
-

5 • The bootloader **BUILT, FIRST RUN PENDING**

Why. Every firmware test today means flashing the device through its update menu. A bootloader that comes first at power-on can instead *start a firmware from the SD card without flashing*, boot the installed firmware as usual, dump the flash to the card, and write update files into the flash from its own menu — with checks the factory updater does not make.

Base and additions. The base is the Freetribe bootloader from the hacktribe community (a small program with display menu, SD card, USB and a debugger connection). On top of it we added: a handoff that recognises which firmware family it is starting and only applies the synth-specific patch there (the unmodified version would crash the sampler firmware at the first sequencer stop); a fix for booting update files from the SD card; a flash installer for all five update file types from the SD browser with header and length checks; and this week's hardening review — the "boot from flash" path now verifies the image before jumping, wrong files can no longer be flashed into the boot sector over USB, every flash wait has a timeout instead of hanging behind "do not power off", and the error messages say what actually happened.

What was verified against the factory parts: memory timings and the boot header are identical to the factory boot loader; the factory boot sector is backed up and can be written back through the normal update menu.

The first run is deliberately volatile: the bootloader is loaded into RAM over the debug connector, shows its menu, boots a firmware from the SD card and boots the installed one — nothing is written. Only after that does "install" become a decision.

6 • Status at a glance

AREA	STATE	NEXT MILESTONE
TekkForge editor, generator, banks, effects, device tools	WORKS	listening tests of the newest sets; hidden device switches
Omnitribe add-on: talk, read, set, inject, memory access	WORKS	—
Omnitribe modules: load, place, run, callbacks	WORKS	—
Omnitribe event routing: sound out, timer tick	WORKS	add a hook for external MIDI notes/controllers (system-exclusive already arrives)
Omnitribe standalone (no computer), pads, on-device pages	PLANNED	main-loop hook, then modules built into the firmware
Bootloader with SD boot and flash installer	BUILT	volatile run over the debug connector, then install decision
Oscillator table +88 (OSZ88), crossgrade	WORKS	—
Extended modulation types 72–131 (MOD132)	WORKS	settable/audible/named on the device via the Omnitribe add-on (panel knob still stops at 72)

7 • How we work

Evidence, not belief

Every claim carries its evidence level: measured on the device, verified on the computer, or planned. Two false leads this year were exactly that — believed instead of measured.

Reversible steps

New firmware functions run in memory first; a power cycle restores the plain device. Permanent installation is a separate, explicit decision.

Backups before writing

The flash is fully backed up, the factory boot sector is saved, and every file the app overwrites keeps twenty previous versions.

Tested automatically

More than 7,500 automated tests run on every change across both projects, plus scripted runs in the real application with screenshots.

Local by default

Analysis, separation and generation run on the user's machine. Only the link import and the optional AI request talk to the outside — and only when used.

Credits and licences

Part of the device knowledge comes from the hacktribe and Freetribe projects; sources and licence terms are documented. Modified firmware images are derived works of the manufacturer's firmware and are not distributed.

TekkForge & Omnitribe — overview and current state, 18 September 2026. Detailed technical documentation, test plans and reverse-engineering notes live in the two repositories.